

On this page

Azul Platform Prime

[Azul Platform Prime](#) is a highly optimized JVM and elastic runtime that breaks traditional Java scale barriers and provides applications with improvement in Java responsiveness, scale, and throughput.

Why Azul Platform Prime?[a](#)[u](#)[i](#)[o](#)

Pulse (real-time) is a memory intensive application due the way the real-time metrics are stored / calculated. Metrics are kept in memory and for each transaction the metrics for the entities involved (eg. Customer) are updated (according to the windows definitions transactions need to be expired, windows updated and metrics calculated) - and this can be a memory and GC intensive task depending on the number of transactions, metrics, size of the windows, etc).

Azul prime optimizes these kinds of workloads where low-latency / high-throughput with high memory usage is needed mainly due to its pauseless garbage collector.



Feedzai recommends the usage of Azul Prime (in Pulse Real Time machines) for the most demanding use cases (in terms of volumes). In those cases Azul Prime is required to accomplish the latencies and the throughput required.

Azul Platform Prime Installation[a](#)[u](#)[i](#)[o](#)



Azul Platform Prime is only required for Pulse RT nodes.

Azul Platform Prime can be downloaded through the [Azul website](#)

Pulse supports the last stable build compatible with the JDK8 (at the time of this document was written 22.08.301.0).

Alternatively Feedzai is also providing the packages in nexus:

- zing [rpm](#) [zip](#) [tar.gz](#)
- zing-zst [rpm](#) [zip](#) [tar.gz](#)



As the links to download the previous binaries from the Azul website are behind an account required link, Feedzai is providing the installation packages in [nexus](#) - since it was previously approved and configured as the standard way to provide Feedzai software to HSBC.

Due security guardrails, HSBC was having problems downloading the `RPM` packages, it was requested if it is possible to provide the packages in `zip` and `tar.gz` formats. This is the reason why there are multiple links for the Azul installation files.

To install the software please follow the steps on [this page](#).

Validate the installation[a](#)[u](#)[i](#)[o](#)

Execute `java -version` or `/opt/zing/zing-<jdk-version>/bin/java -version` (if you don't have the java command on path):

The output should be similar to the one below:

```
java version "1.8.0-zing_19.07.0.0"  
Zing Runtime Environment for Java Applications (build 1.8.0-zing_19.07.0.0-b3)  
Zing 64-Bit Tiered VM (build 1.8.0-zing_19.07.0.0-b4-product-linux-X86_64, mixed mode)
```

Copy

License installation

The license key is not necessary to run Azul Platform Prime. However, to comply with your internal documents or processes regarding contract compliance, it may be required to install an Azul license key.

To install the licence key on a machine, copy the license file to your target host's `/etc/zing/` directory. To use any other file location, you must add `-XX:ZingLicenseFile=<filename>` to your Azul Zulu Prime JVM instance launch command. Licenses allow Azul Zulu Prime JVM instance launches up to the end of the day they are due to expire within the local timezone specified in the license file.

To force Azul Prime (version 20.10 and newer) to verify the license, add the following arguments: `-XX:+UseCommercialFeatures -XX:+ValidateLicenseKey`.

Some applications depend on the `JAVA_HOME` environment variable being set. So it is recommended to define the `JAVA_HOME` environment variable to point to the Azul Platform Prime installation directory, e.g. `$ export JAVA_HOME=/opt/zing/zing-<jdk-version>`.

Configurations

The Azul Zulu Prime JVM recognizes the standard and non-standard command line options for the HotSpot VM that is a core component of Java Platform, Standard Edition (Java SE). If the Azul Zulu Prime JVM detects a HotSpot VM native command line option that it does not support, its response can be set with the following option:

```
-Xnativevmflags:[ignore|error|warn]  
[source, shell,opts="user,check"]
```

Copy

The default option is ignore.

This is important because it allows us to test the Azul Prime installation without changing the already defined JVM parameters.



One initial test to check if it is everything ok can be simply change the JVM and run everything else as usual.

Azul Platform Prime Memory allocation

Azul Platform Prime uses the Azul Zulu Prime JVM to run Java applications, similar to a typical JVM running a Java application. The significant difference is how memory is handled. Azul Zulu Prime JVM uses the Azul Zulu Prime System Tools (ZST) for managing the memory of three components: Contingency memory, Pause Prevention memory, and Zing Memory, the memory used for the Azul Zulu Prime JVM's Garbage Collector.

Using Azul Zulu Prime JVM with the Garbage Collector and Contingency memory means that your Java application is no longer limited to the maximum heap size specified on the Java command line when you started your Java application. As an additional resource, Azul Zulu Prime JVM's Garbage Collector might temporarily use Pause Prevention memory to prevent pauses that are caused by lack of available clean pages.

Azul Platform Prime provides two different methods to configure/instruct JVM to use the system memory:

- **Reserve-at-config:** memory for use by the JVMs running on the machine can be reserved at configuration
- **Reserve-at-launch:** the memory reservation for each JVM can be deferred until the launch of that JVM process

For more details about JVM memory allocation please refer to [Azul zing documentation](#).

Feedzai recommends reserved-at-config. On a typical production installation, around 80 % of the total system memory is reserved for zing, leaving 20 % for the OS/Kernel and/or off-heap allocation. Out of 80%, 75 % is set for Reservable Memory where all the applications JVMs will run, and the remaining ~4% are divided equally between Pause Prevention Memory and Contingency Memory.

For more information about how to set up the memory please check [this guide](#).

Monitoring📊

Pulse already reports information about the JVM status by showing for instance the heap usage, garbage collector periodicity and execution time, and number of threads via the exposed monitoring endpoints.

The same metrics can be used to monitor the system running on top of Azul Platform Prime.

For more information please check the [Pulse section in the monitoring guide](#).

Additionally Azul also provides an official runtime monitoring and diagnostics tool, for more information please check [this page](#).